CS311 - Design & Analysis of Algorithms

Dr. Maram Alajlan – Office # W357

majlan@psu.edu.sa

Problems and Algorithms

Algorithm: method for solving a problem.

Abstractly, a *problem* is just a function

p : Problem instances -> Solutions

different inputs for the same problem

Example: Sorting a list of integers
Problem instances = lists of integers
Solutions = sorted lists of integers
p : ` L-> Sorted version of L

An *algorithm* for *p* is a program which computes *p*.

There are three related questions which warrant consideration:

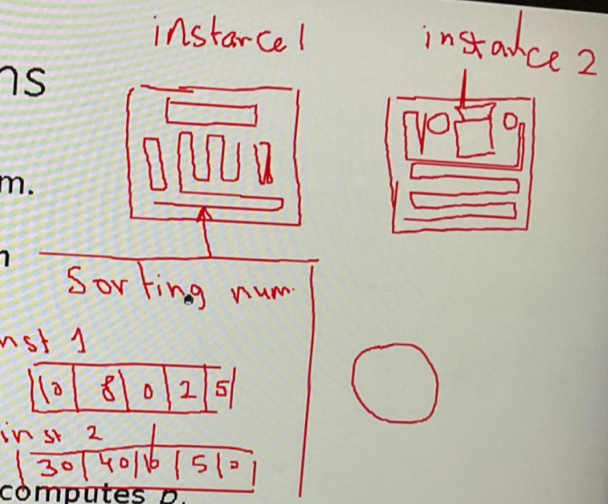
Problems and Algorithms

Algorithm: method for solving a problem.

Abstractly, a *problem* is just a function

p : Problem instances -> Solutions

Example: Sorting a list of integers
Problem instances = lists of integers
Solutions = sorted lists of integers
p : L-> Sorted version of L
An *algorithm* for *p* is a program which computes *p*.



There are three related questions which warrant consideration:

Problems and Algorithms

- **Question 1:** Given an algorithm *A* to solve a problem *p*, how "good" is *A*?

  consider time (takes less time)
  energy (uses less energy)

- **Question 2:** Given a particular problem *p* for which several algorithms are known to exist, which is best?

  A / B / C / D / ...

  IL tests based on the experience / how well an algorithm does

- **Question 3:** Given a problem *p*, how does one design good algorithms for *p*?

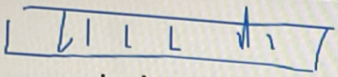
Algorithm Analysis

The main focus of ***algorithm analysis*** in this course will be upon the "quality" of algorithms already known to be **correct**. (finish the task correctly no matter how long it takes)

This analysis proceeds in two dimensions:

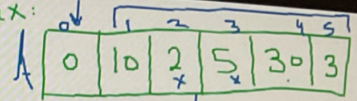
- **Time complexity**: The amount of time which execution of the algorithm takes, usually specified as a function of the size of the input O( )
- **Space complexity**: The amount of space (memory) which execution of the algorithm takes, usually specified as a function of the size of the input. (won't be focused much on this course)

Such analyses may be performed both ***experimentally*** and ***analytically***.



Algorithm Analysis

Finding Max:



y = 0
y = 10
y = 30

y = first number
for (i=1 ; i<size ; i++)
if y < A[i]
y = A[i]

The main focus of ***algorithm analysis*** in this course will be upon the "quality" of algorithms already known to be **correct**.

This analysis proceeds in two dimensions:

The number of oper = n ⇒ O(n)

- **Time complexity:** The amount of time which execution of the algorithm takes, usually specified as a function of the size of the input
- **Space complexity:** The amount of space (memory) which execution of the algorithm takes, usually specified as a function of the size of the input.

Such analyses may be performed both ***experimentally*** and ***analytically***.

Designing Algorithms

It is important to study the process of designing good algorithms in the first place.

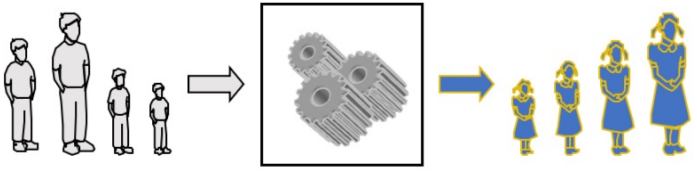
There are two principal approaches to algorithm design.

- **By problem**: Study sorting algorithms, then scheduling algorithms, *etc.*
- **By strategy**: Study algorithms by *design strategy*. Examples of design strategies include:

    Divide-and-conquer
    The greedy method
    Dynamic programming
    Backtracking
    Branch-and-bound

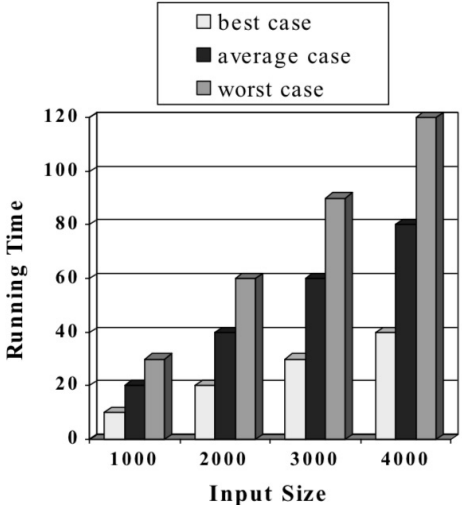
Algorithm Definition

- An algorithm is a step-by-step procedure for solving a problem in a finite amount of time.



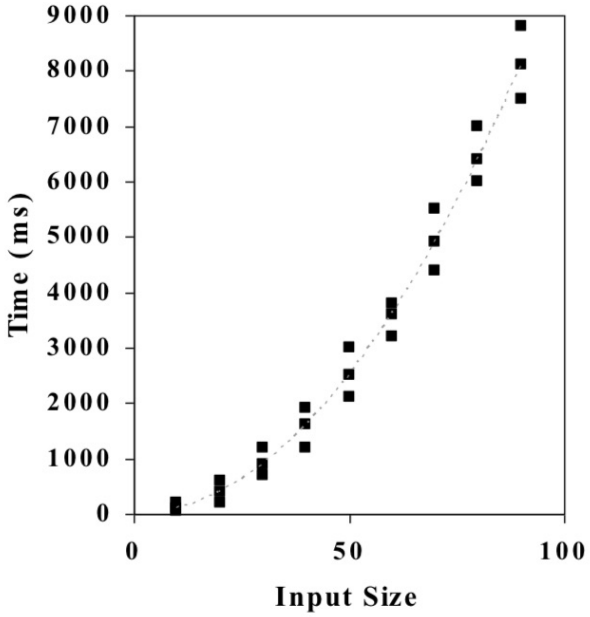
Running Time

- Most algorithms transform input objects into output objects.
- The running time of an algorithm typically grows with the input size.
- Average case time is often difficult to determine.
- We focus on the worst case running time.
  - Easier to analyze
  - Crucial to applications such as games, finance and robotics



Experimental Studies

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition
- Use a method like times() to get an accurate measure of the actual running time
- Plot the results



Limitations of Experiments

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used

Theoretical Analysis

- Uses a high-level description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, n.
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

Search Problem

- Problem: Is the key x in the array S of n keys?
- Inputs(parameters): integer n, array of keys indexed from 1 to n

Sequential Search

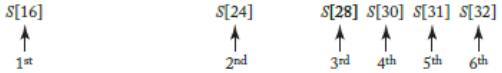
```
void seqsearch (int n,
                const keytype S [ ],
                keytype x,
                index & location)

{
location = 1;
while (location <= n && S[location] != x)
      location ++;
    if (location > n)
       location=0;
}
```

Binary Search

```
void binsearch (int n,
                const keytype S[],
                keytype x,
                index& location)
{
  index low, high, mid;

  low = 1; high = n;
  location = 0;
  while (low < = high &&  lo
     mid = ⌊(low + high)/2⌋;
     if (x = = S[mid])
        location = mid;
     else if (x < S[ mid ])
        high = mid - 1;
```



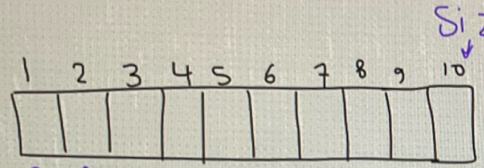
The array items that Binary Search compares with x when x is larger than all the

Sequential Search Versus Binary Search

● **Table 1.1** The number of comparisons done by Sequential Search and Binary Search when x is larger than all the array items

Array Size	Number of Comparisons by Sequential Search	Number of Comparisons by Binary Search
128	128	8
1,024	1,024	11
1,048,576	1,048,576	21
4,294,967,296	4,294,967,296	33

EduHub | Inbox (4,891) - majlan@psu.edu | Untitled – FigJam
figma.com/board/JXQlWKAEvjJmTMNCH6kJHY/Untitled?node-id=0-1&p=f&t=S7cmMwijF3oP38Vi-0
Free



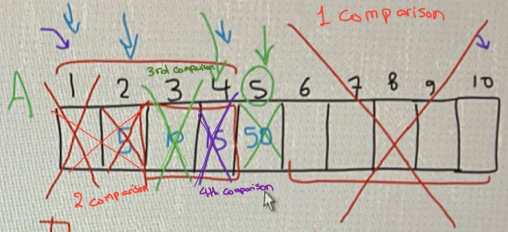
Seq. search

X = 20

Worst case: 10 Comp.

(n)

best case: O(1)



$\log_2 10 = 3.32 \approx 4$ comparisons

Binary Search.

X = 20

Find middle. $\frac{(1+10)}{2} = \frac{11}{2} = 5$ (low + high)

Compare x and A[mid] (A[5])

Find middle of the left side.

$\frac{(1+4)}{2} = \frac{5}{2} = 2.5 = 2$

Find middle $\frac{(3+4)}{2} = \frac{7}{2} = 3.5 = 4$ (low + high) A[2]

$\frac{4+4}{2} = 4$

worst case: O(log n)

best case: O(1)

Pseudocode (like code)

- High-level description of an algorithm
- More structured than English prose
- Less detailed than a program
- Preferred notation for describing algorithms
- Hides program design issues

Example: find max element of an array

Algorithm ***arrayMax***(***A***, ***n***)
  Input array ***A*** of ***n*** integers
  Output maximum element of ***A***

  currentMax ← ***A***[0]
  for ***i*** ← 1 **to** ***n*** − 1 **do**
    if ***A***[*i*] > ***currentMax*** **then**
      currentMax ← ***A***[***i***]
  return ***currentMax***

Pseudocode Details

- Control flow
  - **if** … **then** … [**else** …]
  - **while** … **do** …
  - **repeat** … **until** …
  - **for** … **do** …
  - Indentation replaces braces

- Method declaration

  Algorithm method (arg [, **arg**…])

  Input …

  Output …

- Method call

  var.method (arg [, **arg**…])

- Return value

  return expression

- Expressions
  - ← Assignment (like = in Java)
  - = Equality testing (like == in Java)
  - n2 Superscripts and other mathematical formatting allowed

Primitive Operations

- Basic computations performed by an algorithm
- Identifiable in pseudocode
- Largely independent from the programming language
- Exact definition not important (we will see why later)
- Assumed to take a constant amount of time in the RAM model

- Examples:
  - *Evaluating an expression*
  - *Assigning a value to a variable*
  - *Indexing into an array*
  - *Calling a method*
  - *Returning from a method*

Counting Primitive Operations

- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Algorithm *arrayMax*(*A*, *n*)	# operations
currentMax ← *A*[0]	2
for *i* ← 1 **to** *n* − 1 **do**	2 + *n*
if *A*[*i*] > *currentMax* **then**	2(*n* − 1)
currentMax ← *A*[*i*]	2(*n* − 1)
I <- I +1 { increment counter ***i*** }	2(*n* − 1)
return *currentMax*	1
	Total $7n - 1$

Estimating Running Time

- Algorithm ***arrayMax*** executes $7n - 1$ primitive operations in the worst case. Define:
  - a = *Time taken by the fastest primitive operation*
  - b = *Time taken by the slowest primitive operation*
- Let $T(n)$ be worst-case time of ***arrayMax***. Then
  $$a(7n - 1) \leq T(n) \leq b(7n - 1)$$
- Hence, the running time $T(n)$ is bounded by two linear functions

Growth Rate of An Algorithm

- The rate at which running time increases as a function of input is called Rate of Growth.
- The performance (Runtime) of an algorithm depends on n, where n is the size of the input.
- Analyze the algorithm's efficiency by determining the number of times some basic operations are done as a function of the input size.
- T(n): time complexity of an algorithm is the determination of how many times the basic operation is done for each value of the input size n.

Growth Rate of Running Time (*)

$T(n) = n$

So $T(100) = 100$

$T(200) = 200$

- Let ***T*(*n*)** be worst-case time of ***seqsearch.***

- Changing the hardware/ software environment
  - Affects **T(n)** by a constant factor, but
  - Does not alter the growth rate of **T(n)**

Linear growth means growing at a constant rate. Each step in growth is exactly as much as the previous level.

- The linear growth rate of the running time ***T*(*n*)** is an intrinsic property of algorithm ***seqsearch***

Constant Factors

- The growth rate is not affected by
  - constant factors or
  - lower-order terms
- Examples
  - 102**n** + 105 is a linear function
  - 105**n**2 + 108**n** is a quadratic function

Table 1.3 The quadratic term eventually dominates

n	$0.1n^2$	$0.1n^2 + n + 100$
10	10	120
20	40	160
50	250	400
100	1,000	1,200
1,000	100,000	101,100

Growth Rate

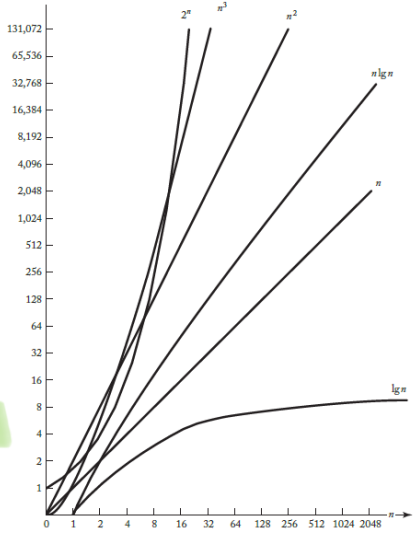
- The algorithms can be classified as:

1. Constant time algorithm O(1)
2. Logarithmic algorithm O(log$_2$ n)
3. Linear O(n)
4. Quadratic O(n2)
5. Polynomial O(nk) O(n^k)
6. Exponential O(kn) O(k^n)

fastest — means the execution time won't affected the input size — halves the input size — slowest

$1 < \log n < \sqrt{n} < n < n \log n < n2 < n3 < \ldots < 2n < .. < n^n$ (2^n, $n!$)

Figure 1.3 Growth rates of some common complexity functions.



Execution times for algorithms with the given time complexities

usually till n^2 its acceptable

n	$f(n) = \lg n$	$f(n) = n$	$f(n) = n \lg n$	$f(n) = n^2$	$f(n) = n^3$	$f(n) = 2^n$
10	0.003 μs*	0.01 μs	0.033 μs	0.10 μs	1.0 μs	1 μs
20	0.004 μs	0.02 μs	0.086 μs	0.40 μs	8.0 μs	1 ms†
30	0.005 μs	0.03 μs	0.147 μs	0.90 μs	27.0 μs	1 s
40	0.005 μs	0.04 μs	0.213 μs	1.60 μs	64.0 μs	18.3 min
50	0.006 μs	0.05 μs	0.282 μs	2.50 μs	125.0 μs	13 days
10^2	0.007 μs	0.10 μs	0.664 μs	10.00 μs	1.0 ms	4×10^{13} years
10^3	0.010 μs	1.00 μs	9.966 μs	1.00 ms	1.0 s	
10^4	0.013 μs	10.00 μs	130.000 μs	100.00 ms	16.7 min	
10^5	0.017 μs	0.10 ms	1.670 ms	10.00 s	11.6 days	
10^6	0.020 μs	1.00 ms	19.930 ms	16.70 min	31.7 years	
10^7	0.023 μs	0.01 s	2.660 s	1.16 days	31,709 years	
10^8	0.027 μs	0.10 s	2.660 s	115.70 days	3.17×10^7 years	
10^9	0.030 μs	1.00 s	29.900 s	31.70 years		

*1 μs = 10^{-6} second.

†1 ms = 10^{-3} second.

Growth Rates

- Growth rates of functions:
  - Linear **n**
  - Quadratic **n2**
  - Cubic **n3**
- The slope of the line corresponds to the growth rate of the function

Growth Rates

- Suppose now that we have two algorithms for the same problem and that their time complexities are $100n$ for the first algorithm and $0.01n2$ for the second algorithm.

$\frac{100n}{0.01n} > \frac{0.01n^2}{0.01n}$

$10000 > n$

$n < 10000$

(slower)

$100(1) > 0.01(1)^2$

$100 > 0.01$

↑ faster

$100(10^5) < 0.01(10^5)^2$

↑ faster

- The first algorithm will **eventually** be more efficient than the second one.

Alg 1 is more efficient when:

$100n < 0.01n^2$

Divide both sides by n (for $n > 0$):

$100 < 0.01n$

Divide by 0.01:

$n > 10000$

So:

- at $n = 10000$ they're equal
- for $n > 10000$, **Alg 1 (100n)** is faster
- for $n < 10000$, **Alg 2 ($0.01n^2$)** is faster

Best Worst and Average Case

- The worst case complexity of the algorithm is the function defined by the maximum number of steps taken on any instance of size n

- The best case complexity of the algorithm is the function defined by the minimum number of steps taken on any instance of size n

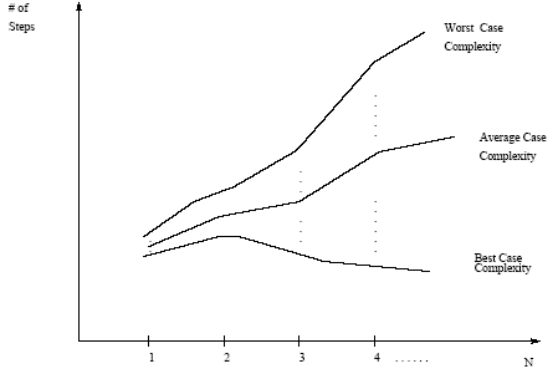
- Each of these complexities defines a numerical function time vs size

Best Worst and Average Case (cont.)

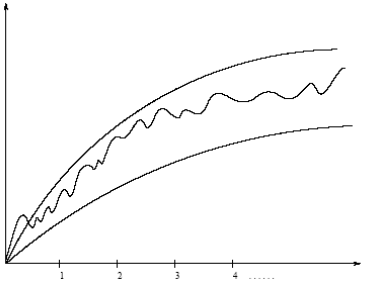
- Best case. Lower bound on cost
  - Determined by "easiest" input.
  - Provides a goal for all inputs.

- Worst case. Upper bound on cost.
  - Determined by "most difficult" input.
  - Provides a guarantee for all inputs.

- Average case. Expected cost for random input
  - Need a model for "random" input.
  - Provides a way to predict performance.



Exact Analysis is Hard !!



- We have agreed that the best, worst and average case complexity of an algorithm is a numerical function of the size of the instances
- However it is difficult to work with exactly because it is typically very complicated
- Thus it is usually cleaner and easier to talk about **upper** and **lower** bounds of the function
- This is where the **Big O notation** comes in

Formalization of the Concept of Order

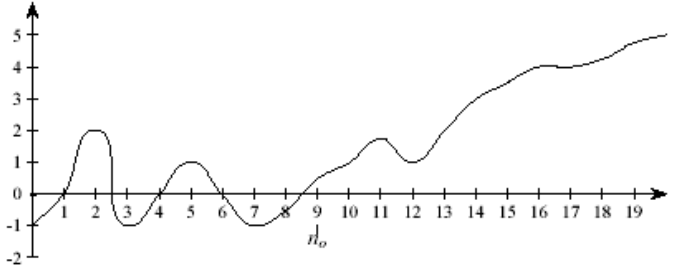
The next task is to formalize the notion of one function being more difficult to compute than another, subject to the following assumptions:

- The argument *n* defining the instance size is sufficiently large
- Positive constant multipliers are ignored.

Formalization of Concepts (cont)

Definition Let *f* :

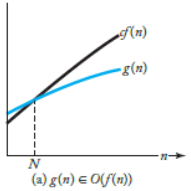
- Complexity functions will never be negative for usable arguments.
- In any case, behavior before *no* is not significant for the asymptotic mathematical analysis.



Commonly-used notations in the theory of algorithms

notation	provides	example	shorthand for	used to
Big Theta	asymptotic order of growth	$\Theta(N^2)$	$\frac{1}{2} N^2$ $10 N^2$ $5 N^2 + 22 N \log N + 3N$ ⋮	classify algorithms
Big Oh	$\Theta(N^2)$ and smaller	$O(N^2)$	$10 N^2$ $100 N$ $22 N \log N + 3 N$ ⋮	develop upper bounds
Big Omega	$\Theta(N^2)$ and larger	$\Omega(N^2)$	$\frac{1}{2} N^2$ N^5 $N^3 + 22 N \log N + 3 N$ ⋮	develop lower bounds

Big-Oh and Growth Rate



- The big-Oh notation gives an **upper bound** on the growth rate of a function
- **The statement "$g(n)$ is $O(f(n))$" means that the growth rate of $g(n)$ is no more than the growth rate of $f(n)$**
- We can use the big-Oh notation to rank functions according to their growth rate

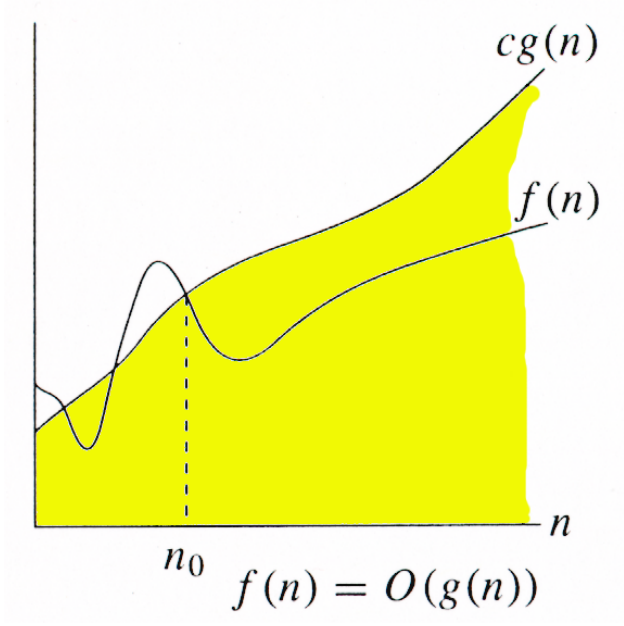
	$g(n)$ is $O(f(n))$	$f(n)$ is $O(g(n))$
$f(n)$ grows more	Yes	No
$g(n)$ grows more	No	Yes
Same growth	Yes	Yes

O-notation

For function $g(n)$, we define $O(g(n))$, **big-O** of n, as the set:

$O(g(n)) = \{f(n)$:
$\exists$ positive constants c and $n0$,
such that $\forall n \geq n0$,
we have $0 \leq f(n) \leq cg(n)\}$

Intuitively: Set of all functions whose *rate of growth* is the same as or lower than that of $g(n)$.



$g(n)$ is an *asymptotic upper bound* for $f(n)$.

$f(n) = \Omega(g(n)) \Rightarrow f(n) = O(g(n))$.
$\Omega(g(n)) \Rightarrow O(g(n))$.

Examples

$O(g(n)) = \{f(n) :$ positive constants c and $n0$, such that $n \geq n0$, we have $0 \leq f(n) \leq cg(n)\}$

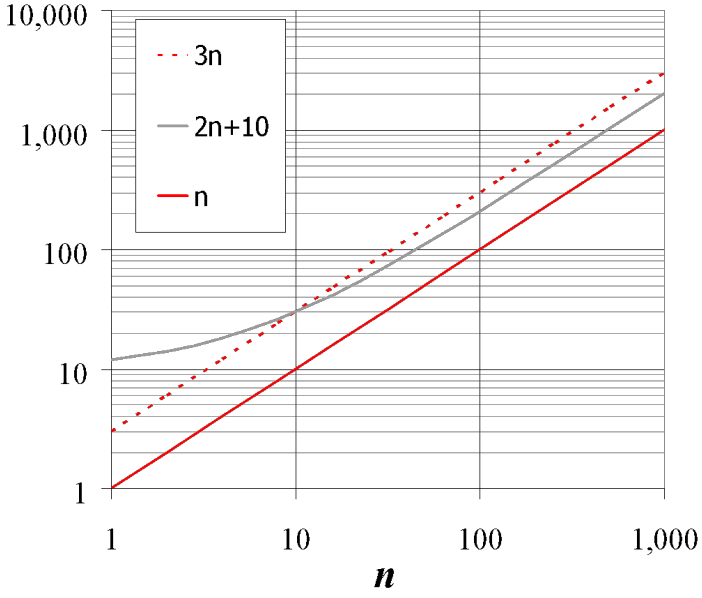
- Show that $3n3=O(n4)$ for appropriate c and $n0$.

Big-Oh Notation

- Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants c and $n0$ such that

  $f(n) \leq cg(n)$ for $n \geq n0$

- Example: $2n + 10$ is $O(n)$
  - $2n + 10 \leq cn$
  - $(c - 2)\, n \geq 10$
  - $n \geq 10/(c - 2)$
  - Pick $c = 3$ and $n0 = 10$



Big-Oh Example

- Example: the function ***n***2 is not ***O***(***n***)
  - **n**2 ≤ **cn**
  - **n** ≤ **c**
  - The above inequality cannot be satisfied since **c** must be a constant

More Big-Oh Examples

- 7n-2

  7n-2 is O(n)

  need c > 0 and n0 ≥ 1 such that 7n-2 ≤ c•n for n ≥ n0

  this is true for c = 7 and n0 = 1

- 3n3 + 20n2 + 5

  3n3 + 20n2 + 5 is O(n3)

  need c > 0 and n0 ≥ 1 such that 3n3 + 20n2 + 5 ≤ c•n3 for n ≥ n0

  this is true for c = 4 and n0 = 21

- 3 log n + log log n

  3 log n + log log n is O(log n)

  need c > 0 and n0 ≥ 1 such that 3 log n + log log n ≤ c•log n for n ≥ n0

  this is true for c = 4 and n0 = 2

Big-Oh Rules

- If is $f(n)$ a polynomial of degree d, then $f(n)$ is $O(nd)$, i.e.,
- Drop lower-order terms
- Drop constant factors
- Use the **smallest** possible class of functions
- Say "2**n** is **O**(**n**)" instead of "2**n** is **O**(**n**2)"
- Use the simplest expression of the class
- Say "3**n** + 5 is **O**(**n**)" instead of "3**n** + 5 is **O**(3**n**)"

Asymptotic Algorithm Analysis

- The asymptotic analysis of an algorithm determines the running time in big-Oh notation
- To perform the asymptotic analysis
  - *We find the worst-case number of primitive operations executed as a function of the input size*
  - *We express this function with big-Oh notation*
- Example:
  - *We determine that algorithm* **arrayMax** *executes at most* $7n - 1$ *primitive operations*
  - *We say that algorithm* **arrayMax** *"runs in* $O(n)$ *time"*
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Asymptotic Notation

- ***O*, Q, W, *o*, w**
- Defined for functions over the natural numbers.
  - **Ex:** $f(n) = \Theta(n^2)$.
  - Describes how $f(n)$ grows in comparison to n^2.
- Define a ***set*** of functions; in practice used to compare two function sizes.

Asymptotic Complexity

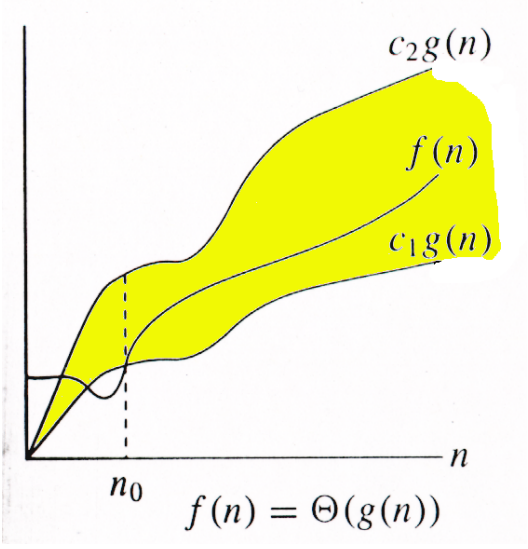
- Running time of an algorithm as a function of input size n **for large n**.
- Expressed using only the **highest-order term** in the expression for the exact running time.
  - Instead of exact running time, say $\Theta(n^2)$.
- Written using ***Asymptotic Notation***.

□-notation

For function $g(n)$, we define □($g(n)$), big-Theta of n, as the set:

□($g(n)$) = {$f(n)$: □ positive constants $c1$, $c2$, and $n0$, such that □n □ $n0$, we have 0 □ $c1g(n)$ □ $f(n)$ □ $c2g(n)$}

Intuitively: Set of all functions that have the same *rate of growth* as $g(n)$.



***f*(*n*) and *g*(*n*) are nonnegative, for large *n*.**

□-notation

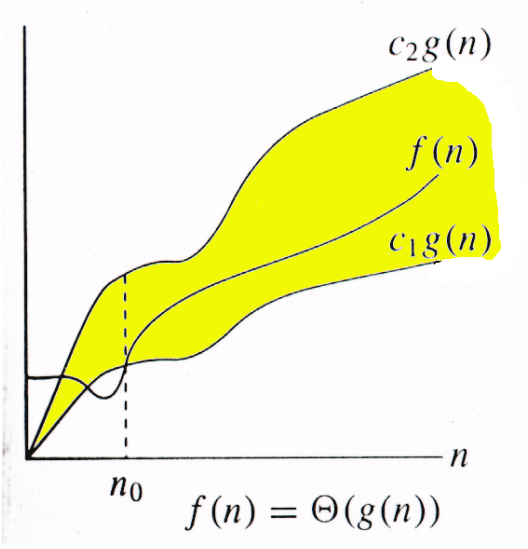
For function $g(n)$, we define □($g(n)$), **big-Theta** of n, as the set:

□($g(n)$) = {$f(n)$: □ positive constants $c1$, $c2$, and $n0$, such that □n □ $n0$, we have 0 □ $c1g(n)$ □ $f(n)$ □ $c2g(n)$}

Technically, $f(n)$ □ □($g(n)$).

Older usage, $f(n)$ = □($g(n)$).

I'll accept either…



***f*(*n*) and *g*(*n*) are nonnegative, for large *n*.**

Example

$\Theta(g(n)) = \{f(n) :$ positive constants $c1$, $c2$, and $n0$, such that $n \geq n0$, $0 \leq c1g(n) \leq f(n) \leq c2g(n)\}$

- $10n2 - 3n = \Theta(n2)$
- What constants for $n0$, $c1$, and $c2$ will work?
- Make $c1$ a little smaller than the leading coefficient, and $c2$ a little bigger.
- ***To compare orders of growth, look at the leading term.***
- Exercise: Prove that $n2/2-3n= \Theta(n2)$

Exercise: Prove that $n2/2-3n = Q(n2)$

-- we must find n0, c1,c2 for this definition that fit the equation:
c1 n2 £ n2/2 - 3n £ c2 n2 " n ³ n0

c1 £ 1/2 - 3/n £ c2 **by dividing by n2**

If n ³ 1 then 1/2 - 3/n £ 1/2 **by making c2 equal to ½**

1/2 - 3/n ³ 1/14 when n ³ 7 (1/2 - 3/n = 0 when n = 6)

So c1 = 1/14, c2 = 1/2, n0 = 7

- Showing that $5n^2 + 3$

We need to find a c a

the following inequal

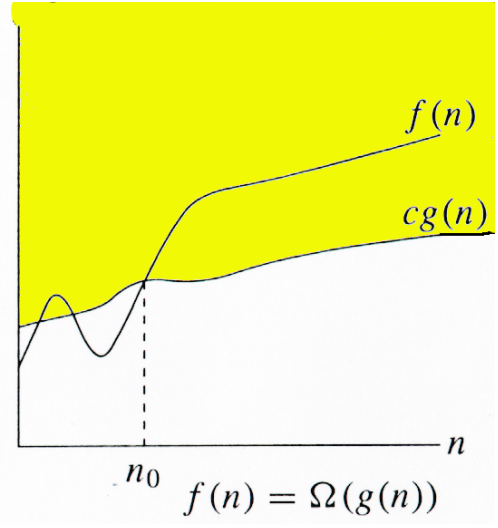
$$5n^2 + 3$$

What would a good c

□ -notation

For function $g(n)$, we define □($g(n)$), **big-Omega** of n, as the set:

□($g(n)$) = {$f(n)$:
□ positive constants c and $n0$,
such that □n □ $n0$,
we have 0 □ $cg(n)$ □ $f(n)$}



Intuitively: Set of all functions whose *rate of growth* is the same as or higher than that of $g(n)$.

$g(n)$ is an *asymptotic lower bound* for $f(n)$.

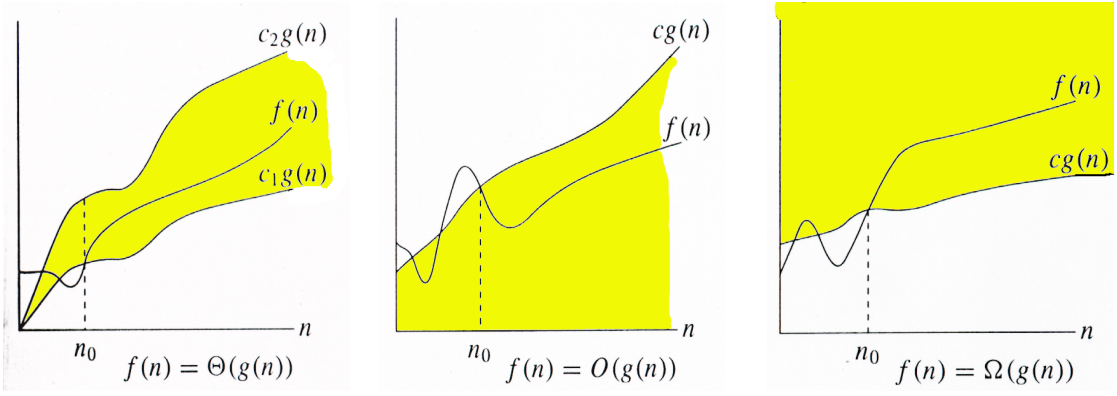
$f(n)$ = □($g(n)$) □ $f(n)$ = □($g(n)$).
□($g(n)$) □ □($g(n)$).

Example

□(*g*(*n*)) = {*f*(*n*) : □ positive constants *c* and *n*0, such that □*n* □ *n*0, we have 0 □ c*g*(*n*) □ *f*(*n*)}

- **1,000,000 n2 □ W (*n*2) why /why not?**
- **(*n* - 1)*n* / 2 □ W (*n*2) why /why not?**
- ***n* / 2 □ W (*n*2) why /why not?**
- **lg (n2) □ W (lg n) why /why not?**
- ***n*2 □ W (*n*) why /why not?**

Relations Between Q, *O*, W



- The value of n0 shown is the minimum possible value (any greater value would also work

Relations Between Q, W, *O*

Theorem : For any two functions $g(n)$ and $f(n)$,
$f(n) = \Theta(g(n))$ iff
$f(n) = O(g(n))$ **and** $f(n) = \Theta(g(n))$.

- I.e., $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$ [intersection]
- In practice, asymptotically tight bounds are obtained from asymptotic upper and lower bounds.

o-notation

For a given function $g(n)$, the set little-*o*:

$o(g(n)) = \{f(n)$: $\forall\, c > 0$, $\forall\, n0 > 0$ such that $\forall\, n \geq n0$, we have $0 \leq f(n) < cg(n)\}$.

$f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity:

$$\lim_{n \to \infty} [f(n) / g(n)] = 0$$

$g(n)$ is an ***upper bound*** for $f(n)$ that is not asymptotically tight.

Observe the difference in this definition from previous ones. **Why?**

ω-notation

For a given function $g(n)$, the set little-omega:

$\omega(g(n)) = \{f(n)$: $\forall\, c > 0$, $\forall\, n0 > 0$ such that $\forall\, n \geq n0$, we have $0 \leq cg(n) < f(n)\}$.

$f(n)$ becomes arbitrarily large relative to $g(n)$ as n approaches infinity:

$$\lim_{n \to \infty} [f(n) / g(n)] = \infty.$$

$g(n)$ is a ***lower bound*** for $f(n)$ that is not asymptotically tight.

Example Uses of the Relatives of Big-Oh

- **5n2 is □(n2)**

  f(n) is □(g(n)) if **there is a constant** c > 0 and an integer constant n0 □ 1 such that f(n) □ c•g(n) for n □ n0

  let c = 5 and n0 = 1

- **5n2 is □(n)**

  f(n) is □(g(n)) if **there is a constant** c > 0 and an integer constant n0 □ 1 such that f(n) □ c•g(n) for n □ n0

  let c = 1 and n0 = 1

- **5n2 is □(n)**

  f(n) is □(g(n)) if, **for any constant** c > 0, there is an integer constant n0 □ 0 such that f(n) □ c•g(n) for n □ n0

  need 5n02 □ c•n0 □ given c, the n0 that satifies this is n0 □ c/5 □ 0

Properties

- **Transitivity**

$f(n) = \Theta(g(n))$ & $g(n) = \Theta(h(n))$ $\Rightarrow$ $f(n) = \Theta(h(n))$

$f(n) = O(g(n))$ & $g(n) = O(h(n))$ $\Rightarrow$ $f(n) = O(h(n))$

$f(n) = \Omega(g(n))$ & $g(n) = \Omega(h(n))$ $\Rightarrow$ $f(n) = \Omega(h(n))$

$f(n) = o(g(n))$ & $g(n) = o(h(n))$ $\Rightarrow$ $f(n) = o(h(n))$

$f(n) = \omega(g(n))$ & $g(n) = \omega(h(n))$ $\Rightarrow$ $f(n) = \omega(h(n))$

- **Reflexivity**

$f(n) = \Theta(f(n))$ $\quad$ $f(n) = O(f(n))$ and $f(n) = \Omega(f(n))$

Relatives of Big-Oh

- big-Omega
  - f(n) is Ω(g(n)) if there is a constant c > 0
    and an integer constant n0 ≥ 1 such that
    f(n) ≥ c•g(n) for n ≥ n0
- big-Theta
  - f(n) is Θ(g(n)) if there are constants c' > 0 and c'' > 0 and an integer constant n0 ≥ 1 such that c'•g(n) ≤ f(n) ≤ c''•g(n) for n ≥ n0
- little-oh
  - f(n) is o(g(n)) if, for any constant c > 0, there is an integer constant n0 ≥ 0 such that f(n) ≤ c•g(n) for n ≥ n0
- little-omega
  - f(n) is ω(g(n)) if, for any constant c > 0, there is an integer constant n0 ≥ 0 such that f(n) ≥ c•g(n) for n ≥ n0

Intuition for Asymptotic Notation

Big-Oh

- f(n) is O(g(n)) if f(n) is asymptotically less than or equal to g(n)

big-Omega

- f(n) is Ω(g(n)) if f(n) is asymptotically greater than or equal to g(n)

big-Theta

- f(n) is Θ(g(n)) if f(n) is asymptotically equal to g(n)

little-oh

- f(n) is o(g(n)) if f(n) is asymptotically **strictly** less than g(n)

little-omega

- f(n) is ω(g(n)) if is asymptotically **strictly** greater than g(n)

Use of limits to determine order of growth.

We can sometimes determine Order using a limit:

We have the following:

$$\lim_{n\to\infty}\frac{g(n)}{f(n)}=\begin{cases} c & \text{implies } g(n)\in\Theta(f(n)) \text{ if } c>0 \\ 0 & \text{implies } g(n)\in o(f(n)) \\ \infty & \text{implies } f(n)\in o(g(n)) \end{cases}$$

Example:

$$\frac{n^2}{2}\in o(n^3)$$

because

$$\lim_{n\to\infty}\frac{n^2/2}{n^3}=\lim_{n\to\infty}\frac{1}{2n}=0.$$

Math you need to Review *

- Summations
- Logarithms and Exponents

- Proof techniques
- Basic probability

- **properties of logarithms:**
  logb(xy) = logbx + logby
  logb (x/y) = logbx - logby
  logbxa = alogbx
  logba = logxa/logxb

- **properties of exponentials**:
  a(b+c) = aba c
  abc = (ab)c
  ab /ac = a(b-c)
  b = a logab
  bc = a c*logab